Node.js — Day 16

URL & Query Parameters

Learning goal: Understand what query parameters are, how they differ from a URL path, and how to read their values in a Node.js server.

> **Day 16 milestone:** You learned to read query parameters using the Node.js **URL** class and **searchParams.get()**.

1. Path vs Query Parameter

A URL can contain a path and additional information after a **?**.

```
/products?category=shoes

/products       → path
category=shoes  → query parameter
```

The **path** identifies the resource. Query parameters provide extra information such as filtering or sorting.

2. Multiple Query Parameters

```
/products?category=shoes&sort=price
```

Here there are two query parameters:

```
category = shoes
sort     = price
```

? starts the query parameters. **&** separates multiple query parameters.

3. Reading Query Parameters in Node.js

```
const http = require("http");

const server = http.createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  console.log(url.pathname);
  console.log(url.searchParams.get("category"));

  res.end("Done");
});

server.listen(3000);
```

For **/products?category=shoes**, this gives:

```
/products
shoes
```

4. url.pathname

```
url.pathname
```

Returns the path portion of the URL.

```
/products?category=shoes
        ↓
pathname → /products
```

5. url.searchParams.get()

```
url.searchParams.get("category")
```

Returns the value of the requested query parameter.

```
/products?category=shoes
          ↓
category → shoes
```

Reusable pattern:

```
url.searchParams.get("parameterName")
```

6. Multiple Parameters in Code

```
const category = url.searchParams.get("category");
const sort = url.searchParams.get("sort");
```

For **/products?category=shoes&sort;=price**:

```
category → "shoes"
sort     → "price"
```

7. Why Query Parameters Matter

Query parameters are common in backend APIs for filtering, sorting, and pagination.

```
/products?category=shoes
/products?category=shoes&sort=price
/users?page=2
```

These are examples of URL structures you will encounter when building REST APIs.

8. Important Rules

• A query parameter is extra information added to a URL after **?**.

• **&** separates multiple query parameters.

• **url.pathname** gives the path.

• **url.searchParams.get("name")** retrieves one parameter value.

• **searchParams** is not called directly like a function; use its **get()** method.

• Query parameters are commonly used for filtering, sorting, and pagination.

9. Common Mistake — and the Correction

You initially wrote:

```
url.searchParams("category")
url.searchParams("sort")
```

The correct version is:

```
url.searchParams.get("category");
url.searchParams.get("sort");
```

> The key detail is **.get()**: **searchParams** gives access to the parameters, while **get()** retrieves one parameter's value.

10. Request Mental Model

GET /products?category=shoes&sort;=price
↓
req.url
↓
new URL(...)
↓
pathname → /products
searchParams.get("category") → shoes
searchParams.get("sort") → price

11. Practice Completed

Question 1 ■ **page=2** is the query parameter in **/users?page=2**.

Question 2 ■ **url.searchParams.get("category")** returns **shoes**.

Final exercise ■ For **/products?category=shoes&sort;=price**, the values are **shoes** and **price**.

Practical exercise ■ Corrected the missing **.get()** and wrote both retrieval statements.

Day 16 practice completed successfully. ■

12. Quick Revision

Path: /products

Query parameter: category=shoes

Multiple parameters: category=shoes&sort;=price

Path property: url.pathname

Parameter value: url.searchParams.get("category")

Key Takeaway

Query parameters let a client send extra information in a URL. In Node.js, the URL class and searchParams.get() make it easy to read those values. This becomes very useful when we build real REST APIs.